

บทที่ 6

การพัฒนาระบบไฟร์วอลล์

6.1 แนวคิด

ระบบไฟร์วอลล์เป็นระบบสำหรับเป็นไฟร์วอลล์ส่วนบุคคล ที่มีระบบตรวจจับผู้บุกรุกทางเครือข่ายคอมพิวเตอร์ที่พัฒนาขึ้นเองโดยภาษา C เพื่อใช้งานร่วมกับไฟร์สแตนด์ในการรับกฎของเพอร์ชันนอลไฟร์วอลล์สำหรับแต่ละเครื่องลูกข่าย โดยเป็นการใช้งานไฟร์วอลล์แบบอิงผู้ใช้คือเมื่อผู้ใช้งานมีการล็อกอินใช้งานเครื่องที่อยู่ในระบบเครือข่ายภายในเครื่องใดก็ตาม ก็จะมีกฎสำหรับผู้ใช้คนนั้นในการใช้งานเพอร์ชันนอลไฟร์วอลล์เสมอ

6.2 ขอบเขตและความสามารถ

ในระบบจะมีเครื่องลูกข่ายหลายเครื่องโดยทุกเครื่องจะมีการติดตั้งไฟร์วอลล์ส่วนบุคคล ซึ่งเป็นเอเจนต์ที่ฝังตัวทำงานอยู่แบบอัตโนมัติ โดยจะมีการควบคุมดูแลจากส่วนกลาง ทำให้ผู้ใช้ไม่รับรู้ถึงการทำงานของเอเจนต์ รวมไปถึงผู้ดูแลระบบสามารถกำหนดกฎการป้องกันการบุกรุกให้ผู้ใช้ในแต่ละกลุ่มตามความเหมาะสม

อีกทั้งยังมีส่วนของการทำงานที่เป็นระบบตรวจจับผู้บุกรุกที่คอยทำหน้าที่ตรวจสอบว่ามีการบุกรุกเข้ามาที่เครื่องลูกข่ายนั้นๆหรือไม่ ถ้าจะมีการแจ้งเตือนไปยังเครื่องเซิร์ฟเวอร์กลางเพื่อนำผลที่ได้ไปวิเคราะห์หาแนวทางป้องกันต่อไป และมีส่วนตรวจการเชื่อมต่อเครือข่ายของโปรเซสต่างๆ เพื่อให้ผู้ใช้งานได้รับรู้ว่าโปรเซสอะไรที่เชื่อมต่อกับเครือข่ายอยู่บ้าง และสามารถกำจัดโปรเซสที่น่าสงสัยได้ด้วยตนเอง ทำให้ผู้ใช้สามารถป้องกันตนเองได้ในระดับหนึ่ง

6.3 เทคโนโลยีที่เกี่ยวข้องและการพัฒนาโปรแกรม

การพัฒนาโปรแกรมไฟร์วอลล์จะใช้ Firewall-Hook Driver ซึ่งเป็น DDK (Driver Development Kit) ของทางบริษัทไมโครซอฟต์ ซึ่งไดร์เวอร์ตัวนี้ใช้ในการฟิลเตอร์แพ็คเก็ต การเขียนโปรแกรมนั้นจะใช้เฮดเดอร์ (Header) ที่ชื่อว่า IpDrvFw.h โดยภายในเฮดเดอร์ (Header) นี้จะมีโครงสร้างต่างๆที่ใช้ในการฟิลเตอร์ และใช้ WinPCap ช่วยในการดักจับแพ็คเก็ตเพื่อมาวิเคราะห์หาว่ามีการบุกรุกระบบเกิดขึ้นหรือไม่ โดยกฎที่ใช้ในการฟิลเตอร์นั้นจะรับมาจากฐานข้อมูลของแอ็คทีฟไดเรกทอรี (Active Directory Database)

เทคโนโลยีอื่นๆ ที่ใช้ในการพัฒนาโปรแกรมนี้นี้

6.3.1 Windows Service

ไมโครซอฟต์วินโดวส์เซอร์วิส หรือก่อนหน้านี้ถูกเรียกว่า NT service ซึ่งวินโดวส์เซอร์วิสนี้จะช่วยในการสร้างโปรแกรมที่รันในส่วนของวินโดวส์เซสชัน (Windows session) ซึ่งโปรแกรมที่เป็นเซอร์วิสจะการทำงานตลอดตั้งแต่วินโดวส์เริ่มบูตและทำงานเป็นแบ็คกราวไปเรื่อยๆ จนกว่าจะทำการปิดวินโดวส์ ซึ่งเซอร์วิสจะสามารถควบคุมได้เช่น หยุดการทำงานชั่วคราว (pause) หรือสั่งให้เริ่มการทำงานใหม่ (restart) โดยการควบคุมจะควบคุมผ่าน Services Control Manager ซึ่งเป็นตัวควบคุมเซอร์วิสทั้งหมด โดยทั่วไปโปรแกรมที่เป็นเซอร์วิสจะไม่มีส่วนติดต่อกับผู้ใช้ (User Interface)

6.3.1.1 การสร้างโปรแกรมเซอร์วิส (Service program)

ในการสร้างในที่นี้จะใช้ภาษา C++ ซึ่งจะต้องลง platform sdk ก่อน และทำการ `#include <windows.h>` ด้วย โดยมีขั้นตอนหลักๆดังนี้

ติดต่อกับ SCM

ในการทำงานที่เกี่ยวกับเซอร์วิสทุกอย่าง จะต้องทำการติดต่อกับ SCM (Service Control Manager) เพื่อทำการเปิด database ของ SCM ทุกครั้ง และนำ handle ที่ได้ไปใช้ในการสร้าง, ควบคุมเซอร์วิส โดยในการติดต่อกับ SCM ทำได้ดังนี้

```
SC_HANDLE schSCManager;    // Open a handle to the SC Manager database.

SchSCManager = OpenSCManager (
    NULL,                    // local machine
    NULL,                    // ServicesActive database
    SC_MANAGER_ALL_ACCESS); // full access rights
```

สร้างเซอร์วิส

ทำการสร้างเซอร์วิส โดยนำ handle ของ SCM database มาใช้กับฟังก์ชัน `CreateService` และทำการเซตค่าพารามิเตอร์ต่างๆ ดังนี้

```
SC_HANDLE schService = CreateService (
    schSCManager,            // SCManager database
    TEXT("Sample_Srv"),     // name of service
    lpzDisplayName,         // service name to display
    SERVICE_ALL_ACCESS,     // desired access
    SERVICE_WIN32_OWN_PROCESS, // service type
```

```

SERVICE_DEMAND_START,      // start type
SERVICE_ERROR_NORMAL,      // error control type
szPath,                      // path to service's binary
NULL,                        // no load ordering group
NULL,                        // no tag identifier
NULL,                        // no dependencies
NULL,                        // LocalSystem account
NULL);                       // no password

```

จากตัวอย่างมีพารามิเตอร์ที่สำคัญอยู่ดังนี้

- พารามิเตอร์ตัวแรกที่ชื่อ schSCManager พารามิเตอร์ตัวนี้ได้มาจากขั้นตอนแรก
- พารามิเตอร์ตัวที่สองที่เป็นฟังก์ชัน TEXT("Sample_Srv") พารามิเตอร์นี้จะนำมาใช้เป็น

ชื่อของเซอร์วิส

- พารามิเตอร์ตัวที่สามที่ชื่อ lpzDisplayName พารามิเตอร์นี้จะนำมาใช้แสดงเป็นชื่อ

เซอร์วิสในโปรแกรม Control panel > Administrative tools > Services

- พารามิเตอร์ตัวที่แปด ที่ชื่อ szPath พารามิเตอร์ตัวนี้เป็นที่อยู่ของโปรแกรมที่ต้องการทำให้มีการทำงานเป็นเซอร์วิส

เมื่อเสร็จสิ้นขั้นตอนนี้เราก็จะได้เซอร์วิสโปรแกรมแล้วแต่ยังไม่ครบถ้วน นั่นคือต้องมีส่วนที่ควบคุมการทำงานของเซอร์วิสและส่วนการนำเซอร์วิสออกอีก

ควบคุมการทำงานของเซอร์วิส (Control Service)

การเริ่มเซอร์วิส (Start Service) ก่อนที่จะสั่งให้เซอร์วิสทำการเริ่มต้นการทำงานนั้นจะต้องทำการเปิดเซอร์วิสที่ต้องการจะสั่งก่อน โดยใช้ฟังก์ชัน OpenService ดังนี้

```

SC_HANDLE schService;

schService = OpenService(

    schSCManager,          // SCM database

    "Sample_Srv",          // service name

    SERVICE_ALL_ACCESS);

```

เมื่อทำการเปิดเซอร์วิสที่ต้องการแล้วเราจะได้ handle ของเซอร์วิสนั้น จากนั้นเราก็นำ handle ที่ได้มาทำการเริ่มการทำงานโดยเรียกฟังก์ชัน StartService ดังนี้

```

StartService(
    schService,    // handle to service
    0,             // number of arguments
    NULL)

```

การหยุดเซอร์วิส (Stop Service) จะต้องทำการเปิดเซอร์วิสก่อนเหมือนดังที่ได้กล่าวมาแล้ว จากนั้นทำการหยุดเซอร์วิสโดยเรียกฟังก์ชัน `ControlService` และทำการส่ง พารามิเตอร์ `SERVICE_CONTROL_STOP` กับ service handle ไป ดังนี้

```
ControlService(schService,SERVICE_CONTROL_STOP,&status)
```

หลังจากทำงานเสร็จให้ปิด handle ของเซอร์วิสดังนี้

```
CloseServiceHandle(schService);
```

การนำเซอร์วิสออก (Uninstall Service)

ทำการเปิดเซอร์วิสที่ต้องการเอาออกโดยใช้ `OpenService` ดังที่ได้กล่าวไปแล้ว หลังจากได้ handle ของเซอร์วิสแล้ว เราจะทำการลบเซอร์วิสได้โดยการเรียกฟังก์ชัน `DeleteService` ดังนี้

```
DeleteService(schService);
```

6.3.2 IPC (Inter Process Communication)

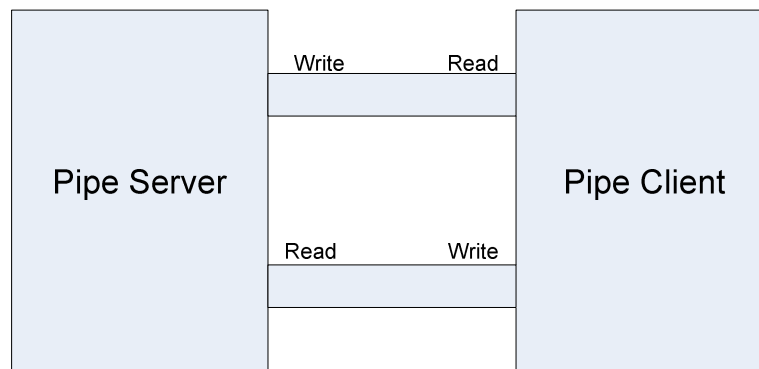
IPC คือกลไกในการติดต่อสื่อสารกันระหว่างโปรเซสหรือระหว่างโปรแกรม โดยปรกติ โปรแกรมที่ใช้กลไก IPC จะใช้ในลักษณะการทำงานของไคลเอนต์เซิร์ฟเวอร์ โดยโปรแกรมที่เป็นไคลเอนต์จะทำการร้องขอไปยังโปรแกรมหรือโปรเซสอื่นๆ และในส่วนโปรแกรมที่เป็นเซิร์ฟเวอร์ จะทำการตอบสนองต่อการร้องขอของโปรแกรมไคลเอนต์ และในหลาย ๆ โปรแกรมอาจจะเป็นทั้งไคลเอนต์และเซิร์ฟเวอร์ในตัวเดียวกันได้ ในการทำ IPC จะมีการทำอยู่หลายวิธีขึ้นอยู่กับลักษณะโปรแกรมที่ใช้ เช่น Clipboard, COM, Data Copy, DDE, File Mapping, Pipes, Windows Socket ... ในที่นี้เราทำ IPC โดยวิธีการทำ Pipes

6.3.2.1 การทำ IPC โดยใช้เทคนิค Pipes

Pipes สำหรับการติดต่อสื่อสารมี 2 ชนิด คือ anonymous pipe และ named pipe ในที่นี้ใช้ named pipe โดยข้อแตกต่างระหว่าง pipes ทั้งสองนี้คือ

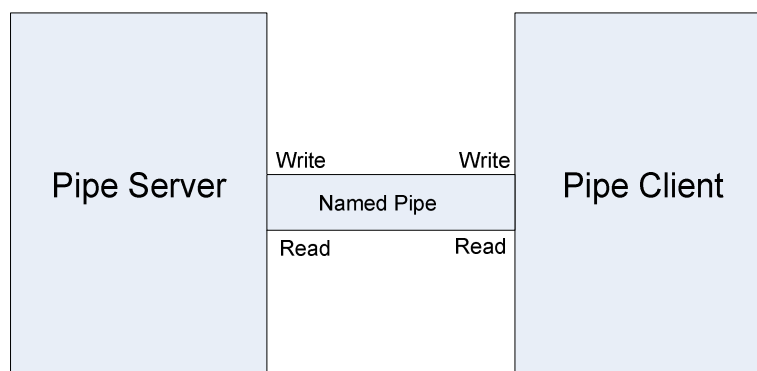
- Anonymous pipe - จะใช้งานในลักษณะที่สองโปรแกรมหรือโปรเซสที่จะทำการติดต่อสื่อสารกันต้องเกี่ยวเนื่องกันเช่นจะต้องเป็น parent process กับ child process โดยการติดต่อทาง pipe server จะทำการสร้าง pipe สำหรับการอ่านและการเขียนขึ้น และทาง

ฝั่งไคลเอนต์จะทำการติดต่อกับเซิร์ฟเวอร์ทางช่องทางนี้โดยไคลเอนต์จะทำการเขียนที่ช่องทางที่ฝั่งเซิร์ฟเวอร์อ่าน และอ่านทางช่องทางที่เซิร์ฟเวอร์เขียน ดังนี้



รูปที่ 6.1 รูปแสดงการทำงานแบบ Anonymous pipe

- Named pipe - จะใช้งานในลักษณะที่สองโปรแกรมหรือโพรเซสไม่จำเป็นต้องเกี่ยวข้องกันโดยการทำงานของ Named pipes จะเริ่มโดย named-pipe server โพรเซสทำการสร้าง named pipe ซึ่งเป็นชื่อของ pipe ที่จะทำการติดต่อกับไคลเอนต์ ซึ่ง named-pipe client จะรู้ชื่อของ pipe ที่ named pipe server สร้างขึ้น แล้วทำการเปิด pipe เพื่อทำการเชื่อมต่อและหลังจากเชื่อมต่อกันแล้ว named pipe client/server จะสามารถส่งข้อมูลหากันได้ ดังรูป



รูปที่ 6.2 รูปแสดงการทำงานแบบ Named pipe

การสร้าง Named pipes ที่ Named pipe server มีขั้นตอนดังนี้

- ที่ฝั่ง server ทำการสร้าง pipe โดยใช้ฟังก์ชัน CreateNamedPipe ดังนี้

```
#define g_szPipeName "\\.\Pipe\MyNamedPipe"

HANDLE hPipe;

hPipe = CreateNamedPipe(
    g_szPipeName,           // pipe name
```

```

PIPE_ACCESS_DUPLEX,           // read/write access
PIPE_TYPE_MESSAGE |           // message type pipe
PIPE_READMODE_MESSAGE |       // message-read mode
PIPE_WAIT,                     // blocking mode
PIPE_UNLIMITED_INSTANCES,      // max. instances
BUFFER_SIZE,                   // output buffer size
BUFFER_SIZE,                   // input buffer size
NMPWAIT_USE_DEFAULT_WAIT,      // client time-out
NULL);                          // default security attribute

```

พารามิเตอร์ตัวแรกคือชื่อ named pipe ที่ทางฝั่งเซิร์ฟเวอร์สร้างขึ้นมาจะมีรูปแบบดังนี้ `\\.\pipe\pipename` ในที่นี้ named pipe มีชื่อเป็น `"\\\\.\\Pipe\\MyNamedPipe"` ซึ่งในส่วนของ pipename อาจจะเป็นชื่อเครื่องก็ได้ในกรณีที่ต้องการติดต่อออกสู่เน็ตเวิร์ก และเมื่อทำการสร้าง named pipe ขึ้นมาเราจะได้ handle ของ pipe มาใช้ในการติดต่อที่นี้ คือ hPipe

- รอการร้องขอจาก Client โดยใช้ฟังก์ชัน `ConnectNamedPipe` ดังนี้

```
//Wait for the client to connect
```

```
BOOL bClientConnected = ConnectNamedPipe(hPipe, NULL);
```

hPipe ที่นี้เป็นพารามิเตอร์คือ handle ของ named pipe ที่ได้จากขั้นตอนแรก

- เมื่อเกิดการร้องขอจากไคลเอนต์ ทางฝั่งเซิร์ฟเวอร์ต้องทำการอ่านข้อความที่ไคลเอนต์ส่งมาใน pipe โดยใช้ฟังก์ชัน `ReadFile` ดังนี้

```
//Read client message
```

```
BOOL bResult = ReadFile(
```

```
hPipe,           // handle to pipe
```

```
szBuffer,        // buffer to receive data
```

```
sizeof(szBuffer), // size of buffer
```

```
&cbBytes,        // number of bytes read
```

```
NULL);           // not overlapped I/O
```

โดยค่าที่อ่านได้จาก pipe จะอยู่ในตัวแปร szBuffer

- เซิร์ฟเวอร์สามารถตอบกลับทางฝั่งไคลเอนต์ได้โดยการเขียน Pipe ดังนี้

```
//Reply to client
bResult = WriteFile(
    hPipe,                // handle to pipe
    szBuffer,             // buffer to write from
    strlen(szBuffer)+1,   // number of bytes to write
    &cbBytes,             // number of bytes written
    NULL);               // not overlapped I/O
```

- เมื่อเสร็จสิ้นการส่งข้อมูลให้ทำการปิด pipe ที่ได้ใช้งานไป ดังนี้

```
CloseHandle(hPipe);
```

การติดต่อ Named pipes ทางฝั่ง Named pipe client มีขั้นตอนดังนี้

ทำการติดต่อกับ server pipe โดยใช้ฟังก์ชัน CreateFile โดยใส่ชื่อ pipe ที่ทางฝั่งเซิร์ฟเวอร์สร้างขึ้นมาเข้าไปที่พารามิเตอร์ตัวแรกในที่นี้คือ g_szPipeName หลังจากนั้นจะได้ handle มาใช้งาน

```
HANDLE hPipe;
hPipe = CreateFile(
    g_szPipeName,        // pipe name
    GENERIC_READ |      // read and write access
    GENERIC_WRITE,
    0,                  // no sharing
    NULL,               // default security attributes
    OPEN_EXISTING,      // opens existing pipe
    0,                 // default attributes
    NULL);             // no template file
```

พารามิเตอร์ตัวที่สอง คือการกำหนดสิทธิในการเขียนหรืออ่าน pipe ในโครงงานนี้ส่วนของไฟร้อลาร์มเอเจนต์จะต้องกำหนดให้มีสิทธิให้อ่านได้เพียงอย่างเดียว เพราะว่าโปรแกรมไฟร้อลาร์มเอเจนต์จะต้องไม่สามารถที่จะเขียนทับ pipe ที่สร้างมาจากวินโดวส์เซอร์วิสได้

- ในส่วนของการอ่านหรือเขียนลง pipe จะทำเหมือนกับฝั่งเซิร์ฟเวอร์
- ปิด handle pipe ทุกตัวเหมือนกับทางฝั่งเซิร์ฟเวอร์

6.4 การทำงานของโปรแกรม

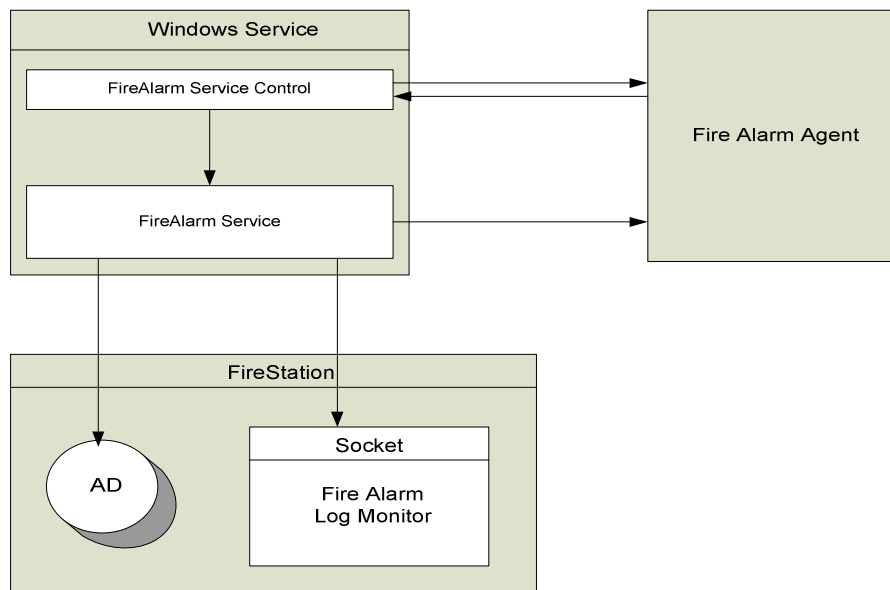
โปรแกรมไฟร้อลาร์มมีลักษณะการทำงานเป็นแบบวินโดวส์เซอร์วิสซึ่งการทำงานลักษณะนี้ระบบจะเป็นตัวรันโปรแกรม ซึ่งจะมีข้อดีดังนี้

- ทำให้ผู้ใช้ไม่จำเป็นต้องมีสิทธิเป็นผู้ดูแลระบบในการเข้าใช้งาน ดังนั้นผู้ใช้จึงไม่สามารถทำการปิดโปรแกรมได้ นอกจากผู้ดูแลระบบเท่านั้น
- ทำให้สามารถป้องกันเครื่องคอมพิวเตอร์ได้แม้ผู้ใช้ยังไม่ได้ทำการล็อกอิน

โปรแกรมไฟร้อลาร์มมีการทำงานหลักๆ แบ่งออกเป็น 3 ส่วนด้วยกันได้แก่

- **ไฟร้อลาร์มเซอร์วิส (Firealarm Service)** ทำหน้าที่ในการดึงข้อมูลจาก AD แล้วทำการฟิลเตอร์ข้อมูลนั้นๆ นอกจากนี้ยังมีส่วนตรวจจับผู้บุกรุกแล้วทำการส่งรายงานไปยังไฟร์สเตชัน
- **ไฟร้อลาร์มเซอร์วิสคอนโทรล (Firealarm Service Control)** ทำหน้าที่ในการควบคุมการทำงานของไฟร้อลาร์มเซอร์วิส เช่น ขั้นตอนของการล็อกอินผู้ใช้
- **ไฟร้อลาร์มเอเจนต์โปรแกรม (Firealarm Agent Program)** ทำหน้าที่แสดงผลของผู้ใช้และแสดงรายงานเมื่อมีผู้บุกรุกเข้ามา และมีระบบตรวจการใช้งานเครือข่ายของโปรเซส

โดยการทำงานของโปรแกรมนี้นี้จะทำงานตั้งแต่เริ่มระบบปฏิบัติการวินโดวส์ โดยเมื่อผู้ใช้ทำการล็อกอินเข้าใช้ระบบ โปรแกรมไฟร้อลาร์มเซอร์วิสคอนโทรล จะทำการตรวจสอบเหตุการณ์การล็อกอินของผู้ใช้ แล้วทำการสั่งให้โปรแกรมไฟร้อลาร์มเซอร์วิสทำการเริ่มการทำงานใหม่เพื่อทำการดึงข้อมูลที่ถูกต้องของผู้ใช้ เมื่อโปรแกรมไฟร้อลาร์มเซอร์วิสพร้อมที่จะใช้งานแล้ว โปรแกรมไฟร้อลาร์มเซอร์วิสคอนโทรลจะสั่งให้โปรแกรมไฟร้อลาร์มเอเจนต์ทำงาน โดยโปรแกรมไฟร้อลาร์มเอเจนต์ทำหน้าที่ในการติดต่อกับไฟร้อลาร์มเซอร์วิส เพื่อดึงข้อมูลของผู้ใช้มาแสดง เมื่อเกิดเหตุการณ์มีผู้บุกรุกเข้ามา โปรแกรมไฟร้อลาร์มเซอร์วิสจะทำหน้าที่ตรวจสอบและทำการรายงานการบุกรุกผ่านทาง Socket ไปยังเครื่องไฟร์สเตชัน โดยมีโปรแกรมไฟร้อลาร์มล็อกมอนิเตอร์เป็นตัวรับรายงานและเก็บลงฐานข้อมูลต่อไป



รูปที่ 6.3 รูปแสดงการทำงานของโปรแกรมไฟร้อลาร์ม

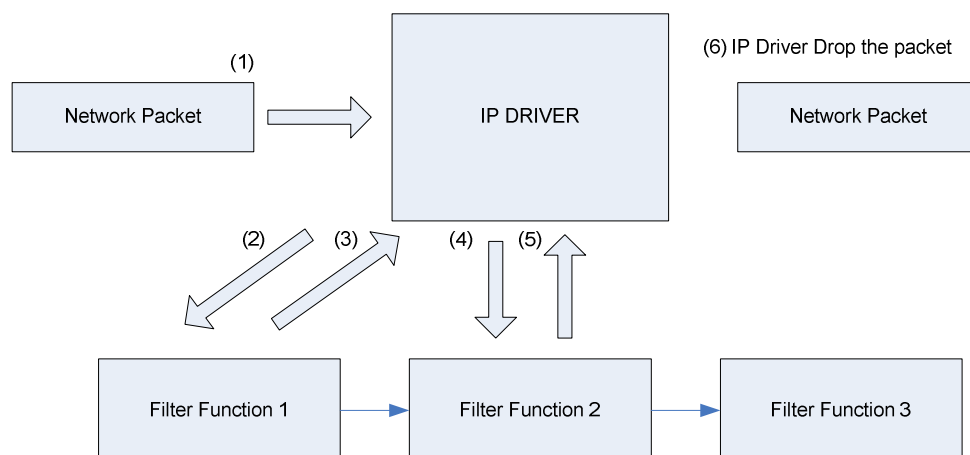
6.4.1 ไฟร้อลาร์มเซอร์วิส (Firealarm Service)

โปรแกรมไฟร้อลาร์มเซอร์วิส มีการทำงานหลักๆ อยู่ 2 ส่วนคือ

- ทำหน้าที่เป็นไฟร้อลส่วนบุคคล
- ทำหน้าที่เป็นระบบตรวจจับผู้บุกรุก

6.4.1.1 แพ็คเก็ตฟิลเตอร์ริงไฟร้อลบนวินโดวส์ XP

เพอร์ชันนอลไฟร้อลที่พัฒนามีลักษณะการทำงานเป็นแบบแพ็คเก็ตฟิลเตอร์ริง (Packet Filtering) โดยจะมีการตรวจสอบแพ็คเก็ตด้วยฟังก์ชันฟิลเตอร์ว่าจะอนุญาตหรือไม่อนุญาตให้แพ็คเก็ตนั้นผ่านไปได้ โดยที่ฟังก์ชันฟิลเตอร์จะมีการกำหนดลำดับไว้แล้วให้ระบบเรียกฟังก์ชันฟิลเตอร์เข้าไปทำงานทีละฟังก์ชันตามลำดับ จนกว่าจะมีฟังก์ชันใดฟังก์ชันหนึ่งส่งค่ากลับเป็น “DROP PACKET” ถ้าฟังก์ชันทั้งหมดส่งค่ากลับเป็น “ALLOW PACKET” แพ็คเก็ตเหล่านั้นจะสามารถผ่านไปได้



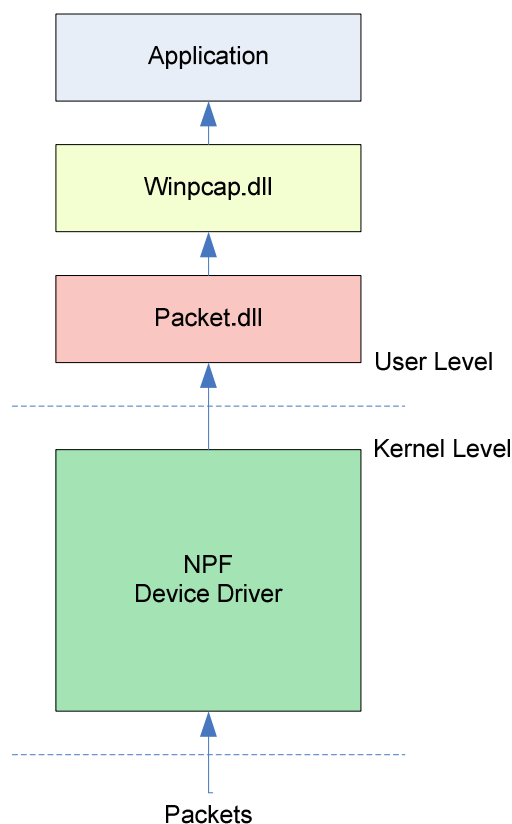
รูปที่ 6.4 แสดงขั้นตอนการทำงานของไฟร้อล

ขั้นตอนการทำงานของไฟร์วอลล์ตามรูปที่ 6.4

- 1) เครื่องได้รับแพ็กเก็ตเข้ามา โดยที่ IP Driver มีฟังก์ชันฟิลเตอร์อยู่ตามที่กำหนดไว้
- 2) IP Driver จะส่งแพ็กเก็ตนั้นเข้าไปยังฟังก์ชันฟิลเตอร์ตามลำดับไปเรื่อยๆ แล้วรอค่าที่ส่งกลับออกมา
- 3) สมมุติฟังก์ชันแรกส่งค่ากลับเป็น “ALLOW PACKET” เมื่อ IP DRIVER ได้รับค่าส่งกลับจากฟังก์ชันแรกเป็น “ALLOW PACKET” ดังนั้น IP DRIVER จะส่งแพ็กเก็ตนั้นไปที่ฟังก์ชันที่สองต่อไปในกรณีนี้
- 4) สมมุติให้ฟังก์ชันที่สองนี้ส่งค่ากลับเป็น “DROP PACKET” เมื่อ IP DRIVER ได้รับค่าส่งกลับจากฟังก์ชันที่สองเป็น “DROP PACKET” ดังนั้น IP DRIVER จะไม่ส่งแพ็กเก็ตนี้ต่อไปยังระบบ และจะไม่ส่งไปยังฟังก์ชันต่อไปอีก

6.4.1.2 ระบบตรวจจับผู้บุกรุกทางเครือข่ายคอมพิวเตอร์

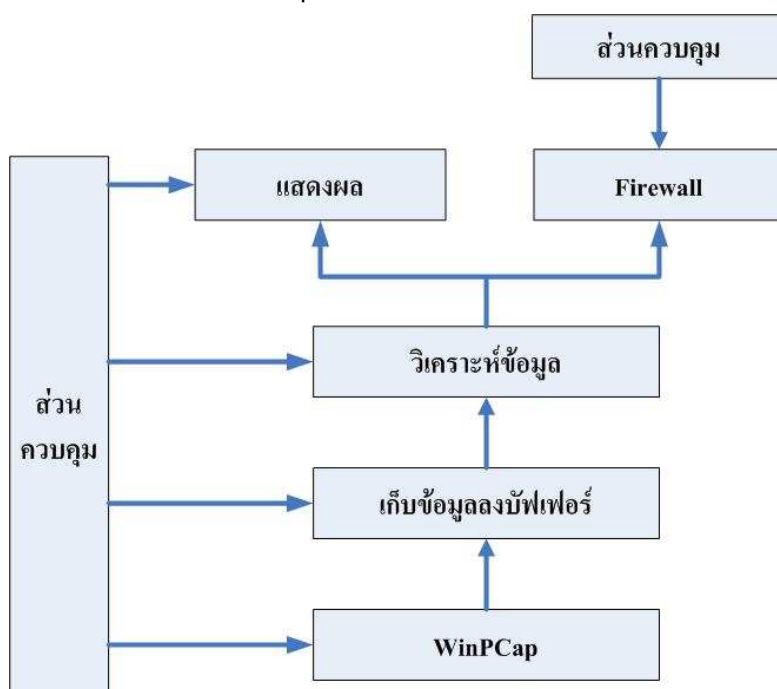
ระบบตรวจจับผู้บุกรุกทางเครือข่ายคอมพิวเตอร์จะอาศัยคุณสมบัติ และ ความสามารถของ WinPCap ซึ่งเป็นไลบรารีที่ทำการติดต่อกับการ์ดแลน เพื่อทำการควบคุมการทำงานของการ์ดแลน และตรวจจับแพ็กเก็ต เพื่อนำมาวิเคราะห์ผลว่ามีการโจมตีเกิดขึ้นหรือไม่



รูปที่ 6.5 แสดงระดับชั้นการทำงานของ WinPCap

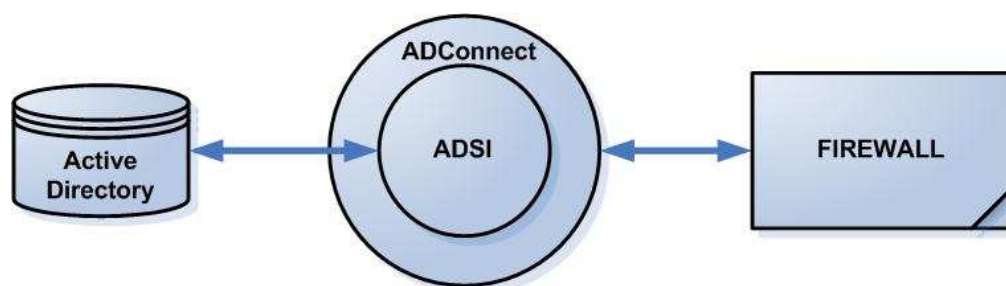
การทำงานของระบบตรวจจับผู้บุกรุกนั้นจะแบ่งการทำงานออกเป็น 2 ส่วนหลักๆ ด้วยกัน คือ ส่วนการตรวจจับแพ็คเก็ตเก็บลงบัฟเฟอร์ และส่วนวิเคราะห์ผลแพ็คเก็ตที่ได้ดักจับมา เมื่อวิเคราะห์ผลเสร็จจะแสดงผล พร้อมกับส่งแพ็คเก็ตไปไฟเตอร์ตามกฎที่กำหนดเอาไว้ดังรูปที่ 6.6

โดยการทำงานทั้ง 2 ส่วนนี้จะมีการแบ่งเชรด (Thread) การทำงานเพื่อที่จะสามารถดักจับแพ็คเก็ตและวิเคราะห์ผลไปได้พร้อมๆ กัน



รูปที่ 6.6 โครงสร้างของระบบตรวจจับผู้บุกรุกทำงานร่วมกับไฟร์วอลล์

เนื่องจากเพอร์ซันนอลไฟร์วอลล์ (Personal Firewall) จะมีความสามารถในการที่จะรับกฎเพื่อเริ่มการทำงานของไฟร์วอลล์จากเครื่องโดเมนคอนโทรลเลอร์ที่ได้ติดตั้งแอ็คทีฟไดเรกทอรีไว้และจะมีการส่งล็อกไฟล์ไปเก็บไว้ยังแอ็คทีฟไดเรกทอรีเมื่อมีการตรวจพบการโจมตี หรือ มีสิ่งผิดปกติเกิดขึ้น ซึ่งโครงสร้างการทำงานของส่วนที่ใช้ในการติดต่อกับเครื่องเซิร์ฟเวอร์แสดงดังรูปที่ 6.7 โดยเพอร์ซันนอลไฟร์วอลล์จะมีการติดต่อไปยังเซิร์ฟเวอร์โดยผ่านคลาส ADConnect ที่สร้างขึ้น



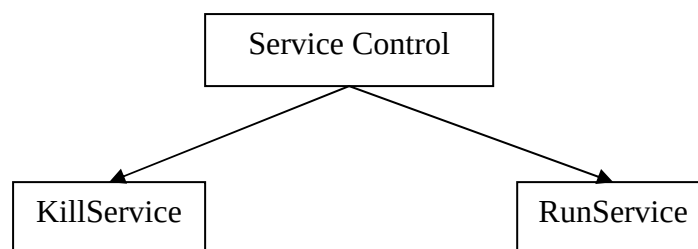
รูปที่ 6.7 แสดงโครงสร้างส่วนติดต่อกับเซิร์ฟเวอร์

เพอร์ชันนอลไฟร์วอลล์จะติดตั้งอยู่ที่เครื่องลูกข่ายทุกเครื่องในระบบโดยมีหน้าที่การทำงานดังนี้

- เป็นไฟร์วอลล์ชนิดแพ็คเกจฟิลเตอร์ริง (Packet Filtering Firewall)
- ตรวจสอบผู้บุกรุกทางเครือข่ายที่เข้ามาที่เครื่องลูกข่าย รวมทั้งสิ่งผิดปกติที่ไม่เป็นไปตามมาตรฐานของโพรโทคอลทีซีพี (TCP) , ยูดีพี (UDP), ไอพี (IP), ไอซีเอ็มพี (ICMP)
- ตรวจสอบการเชื่อมต่อเครือข่ายของโพรเซสต่างๆ โดยผู้ใช้สามารถจบโพรเซสการทำงานและปิดการเชื่อมต่อเครือข่ายของโพรเซสใดๆได้
- ส่งการแจ้งเตือนกลับไปยังล็อกมอนิเตอร์เมื่อตรวจจับได้ว่าการบุกรุกเกิดขึ้น

6.4.2 โปรแกรมไฟร์อลาร์มเซอร์วิสคอนโทรล (FireAlarm Service Control)

ในส่วนของเซอร์วิสคอนโทรลทำหน้าที่ควบคุมการทำงานของเซอร์วิส ประกอบด้วย KillService ซึ่งทำหน้าที่ยกเลิกการทำงานของเซอร์วิส และในส่วน RunService ทำหน้าที่เริ่มต้นการทำงานของเซอร์วิส



รูปที่ 6.8 รูปแสดงส่วนประกอบของ Service Control

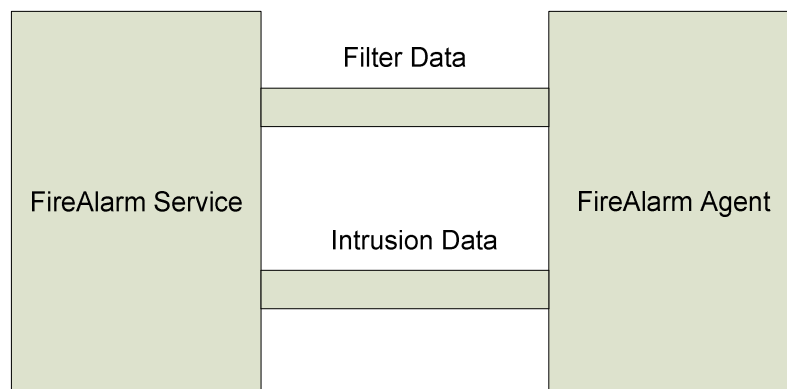
และในการทำงานเมื่อโปรแกรมไฟร์อลาร์มต้องการที่จะทำการอัปเดตข้อมูลใหม่ๆ โปรแกรมไฟร์อลาร์มเซอร์วิสคอนโทรลจะทำการ Refresh เซอร์วิสให้ตัวเซอร์วิสมีการดึงข้อมูลใหม่ๆเข้ามาฟิลเตอร์ โดยการ Refresh จะทำการ KillService ที่ชื่อ “firealarmservice” จากนั้นก็ทำการ RunService อีกครั้ง โดยในการทำ Refresh โปรแกรมไฟร์อลาร์มเซอร์วิสคอนโทรลจะทำการตรวจสอบการล็อกอินเข้าใช้ระบบ และทำการ Refresh เซอร์วิส

6.4.3 โปรแกรมไฟร์อลาร์มเอเจนต์ (Firealarm Agent)

6.4.3.1 ส่วนการรับและแสดงผลข้อมูลที่ได้จากไฟร์อลาร์มเซอร์วิส

ในการรับส่งข้อมูลระหว่างโปรแกรมไฟร์อลาร์มเซอร์วิสกับไฟร์อลาร์มเอเจนต์จะใช้ IPC (Inter Process Communication) ทำการส่งข้อมูล โดยจะต้องทำการสร้าง named pipe server ที่โปรแกรมไฟร์อลาร์มเซอร์วิสและสร้าง named pipe client ที่โปรแกรมไฟร์อลาร์มเอเจนต์ เพื่อทำการรับส่งข้อมูล

โดยในโปรแกรมไฟร์อลาร์มเอเจนต์จะรับข้อมูล 2 อย่างจากไฟร์อลาร์มเซอร์วิส คือข้อมูลกฎการฟิเตอร์และข้อมูลการบุกรุก ในการติดต่อจะทำการสร้าง pipe ขึ้นมา 2 อัน เพื่อทำการส่งข้อมูลกฎการฟิเตอร์และข้อมูลการบุกรุก



รูปที่ 6.9 รูปแสดงส่วนการรับและแสดงผลข้อมูลที่ได้จากไฟร์อลาร์มเซอร์วิส

6.4.3.2 ระบบตรวจการใช้งานเครือข่ายของโปรเซส

ระบบตรวจการใช้งานเครือข่ายของโปรเซสจะทำงานคล้ายกับคำสั่ง `netstat -bn` โดยจะแสดงโปรเซสต่างๆ ที่กำลังใช้งานเครือข่ายอยู่ในขณะนั้น พร้อมทั้งบอกโปรเซสไอดี (PID) โปรโตคอล (Protocol) โลคอลแอดเดรส (Local Address) รีโมตแอดเดรส (Remote Address) และสถานะของการเชื่อมต่อ (State) โดยที่จะมีเวลาการอัปเดตข้อมูลตามเวลาที่ผู้ใช้ได้ตั้งเอาไว้

ระบบตรวจการใช้งานเครือข่ายของโปรเซสนี้มีความสามารถที่จะจบการทำงานของโปรเซส (Kill Connection) และ ปิดการเชื่อมต่อเครือข่ายของโปรเซสนั้นได้ (Close Connection) ถ้าหากพบว่ามีโปรเซสใดที่มีการใช้งานเครือข่ายที่น่าสงสัย ผู้ใช้สามารถจบการทำงานของโปรเซสหรือปิดการเชื่อมต่อได้ทันที เพื่อให้ผู้ใช้สามารถป้องกันตนเองได้อีกทางหนึ่ง